

Himalayan Guardian Nepal - Backend Engineer Technical Assessment

Overview

This is a take-home round for the Backend Engineer position at HGN. You'll get a small, scoped problem close to what our platform actually deals with, and we're mainly checking how you think through it, not just whether the final code runs.

There's no single correct answer here. We want to see your reasoning and how you handle a spec that isn't fully spelled out.

The Problem

HGN tracks trekkers and groups using GPS/satellite tracking devices, and coordinates SOS response when something goes wrong on the trail.

Build a small backend service for SOS alert intake and dispatch coordination that handles the following.

Core Requirements

1. Device sends an alert

A tracking device sends an SOS signal with a device ID, GPS coordinates, and a timestamp. Record it as an alert.

2. One device, multiple orders over time

A device can be tied to more than one active order or booking (reused across treks, reassigned, etc). Your API needs to figure out which order an incoming SOS belongs to when the device isn't cleanly one-to-one with a single order. Don't assume it always is.

3. One device, multiple people in a group

A group booking might share a single device across several trekkers instead of giving everyone their own. Your data model and API should handle an alert from a shared device making sense at the group level.

4. Deduplication

Devices on bad satellite connectivity sometimes retransmit the same SOS multiple times. A retransmission close in time shouldn't create a second alert. A genuinely new SOS should. You decide what counts as "close in time," just explain why you picked that number.

5. Concurrency-safe claiming

A coordinator claims or acknowledges an alert to start responding. Two coordinators should never both end up claiming the same alert. This needs to actually hold up under

concurrent requests, not just look fine when you test it by hand one request at a time.

6. Escalation

If nobody acknowledges an alert within some time window, it should escalate somehow (status change, urgent flag, fallback notification, your call, just be consistent).

Not required

- No frontend. API-only is fine.
 - No real satellite integration. Simulate incoming SOS via API calls or a script.
 - No need for production-grade auth. Focus on the core logic.
-

Deliverables

1. Working code, containerized:

- `Dockerfile` and `docker-compose.yml` so we can run it in one command. This should include a real database (PostgreSQL preferred, since that's what we use, but any relational database is fine) running as its own container, not an in-memory or file-based database. We want to see real transactional/locking behavior, not something that only works because it's single-threaded in memory.
- `.env.example` with required variables (no real secrets committed).
- `README.md` with setup steps and a few example requests.
- Working Swagger/OpenAPI docs (e.g. springdoc-openapi for Spring Boot), generated from the actual code and reachable once the app is running. Not a hand-written doc that's disconnected from the real endpoints.

2. A decision document (`DECISIONS.md`). See below. This matters as much as the code.

3. Submission:

- Create a new **public** GitHub repository named `hgn-assessment-[your-name]`. This should be your own personal repository, not shared with anyone else.
 - Commit as you go rather than pushing everything in one final commit at the end. We'd like to see how the solution came together.
 - Send us the repo link once you're done, no need to add anyone as a collaborator.
-

The Decision Document

Tell us, specifically and in your own words, what you decided for each of these and why. Treat this like explaining your design to a teammate who's about to work in the same codebase, not like writing documentation for a stranger.

- **System design:** how the pieces fit together (services, layers, how a request flows from intake to dispatch), and what you'd change if this had to scale to hundreds of devices sending alerts at once.
- **Data model:** how you represented devices, orders, groups, and alerts, and why you shaped it that way.
- **Device ambiguity:** how you figured out which order or group an alert belongs to when a device isn't one-to-one with an order. What did you assume, and what breaks if that assumption is wrong?
- **Deduplication:** what rule you used to tell a retransmission apart from a new SOS, and why that threshold.
- **Concurrency:** what actually stops two coordinators from claiming the same alert (locking, transactions, a unique constraint, something else), and what you did to check it actually works, not just that it looks right.
- **Escalation:** how it triggers and why you built it that way.
- **Tradeoffs:** where you chose simplicity over robustness (or the other way around), and why, given the time you had.
- **What you're unsure about:** anything you assumed because the spec was ambiguous, and anything you know is still weak or incomplete.

Ground this in your actual code: real table names, real endpoints, and any problems you actually ran into.

On AI Tools

Using AI for boilerplate, syntax lookup, or debugging help is fine. That's normal.

The core design decisions and problem-solving need to be yours. If you use AI assistance for parts of the code, you should still be able to explain and modify all of it without hesitation. Same goes for the decision document: write it yourself, don't have an AI generate it and lightly edit it.

We'll do a short call afterward where you walk us through your code and decisions and answer follow-up questions on the spot. That's the real check.

Timeline

- 4-5 days from receiving this. Should take about 4-6 hours of actual work; it's meant to be small.
- Send your repo link to: hr@hgn.com.np
- Next step: a 20-30 minute call to go through your solution together.